AI Job Displacement Analysis

Building an Interactive Assessment Tool

Magnimind Academy - Mentorship Program

Project Overview

In this project, you will build an interactive tool that analyzes how likely different jobs and tasks are to be replaced or augmented by AI. You will collect occupational data from O*NET, develop a conceptual framework for assessing automation potential, use LLM APIs to analyze jobs, and create a user-friendly application that helps people understand AI's impact on their careers.

This is NOT about using AI to do the work for you. This is about using AI as a tool to analyze jobs systematically based on a framework YOU design.

1 Background and Motivation

The rapid advancement of AI, particularly Large Language Models (LLMs), has sparked intense debate about which jobs will be affected by automation. While some predict widespread job displacement, others argue that AI will augment rather than replace human workers. The reality is nuanced: some tasks within jobs are highly automatable, while others require uniquely human capabilities.

Your challenge is to move beyond speculation and build a **data-driven, systematic framework** for analyzing AI's impact on specific occupations and tasks.

1.1 Why This Matters

- **For Workers:** Understanding which skills to develop and which roles are more resilient
- **For Employers:** Strategic workforce planning and identifying where AI can augment work
- **For Educators:** Designing curricula that prepare students for the AI era
- **For Policymakers:** Creating evidence-based policies for workforce transition
- **For You:** Applying AI practically while thinking critically about its societal impact

2 Learning Objectives

Learning Objectives

By completing this project, you will:

1. **Design conceptual frameworks:** Develop a structured model for assessing automation potential based on job characteristics
2. **Work with real occupational data:** Extract and analyze data from O*NET's comprehensive database
3. **Master prompt engineering:** Design effective prompts that leverage LLMs for systematic analysis
4. **Build practical applications:** Create an interactive tool that non-technical users can use
5. **Think critically about AI:** Understand AI's capabilities and limitations in job displacement
6. **Communicate insights:** Present complex analysis through clear visualizations and explanations
7. **Document decision-making:** Justify every major design choice with evidence and reasoning

3 Data Source: O*NET Database

3.1 What is O*NET?

O*NET (Occupational Information Network) is the U.S. Department of Labor's comprehensive database containing information on hundreds of occupations. It provides standardized data on:

- **Job Tasks:** Specific activities workers perform
- **Skills:** Developed capacities that facilitate learning
- **Abilities:** Enduring attributes of the individual
- **Knowledge:** Organized sets of principles and facts
- **Work Activities:** Types of work behaviors
- **Work Context:** Physical and social factors
- **Tools & Technology:** Equipment and software used

3.2 Accessing O*NET Data

Database Location: https://www.onetcenter.org/database.html

You have two options for accessing the data:

1. **Download Database (Recommended):**
   - Go to https://www.onetcenter.org/database.html
   - Download "O*NET Database" (Latest version, MySQL or Excel format)
   - Contains complete data for all occupations
   - Allows offline analysis
2. **Web Services API:**
   - Use O*NET Web Services: https://services.onetcenter.org/
   - Requires free registration for API credentials
   - Good for real-time lookups
   - Rate limited

3.3 Key Data Files to Focus On

When you download the database, focus on these files:

File	Contains
`Occupation Data.txt`	Basic occupation information (title, description, code)
`Task Statements.txt`	Specific tasks performed in each occupation
`Skills.txt`	Skills required for each occupation with importance levels
`Abilities.txt`	Abilities required with importance and level ratings
`Work Activities.txt`	Generalized work activities with importance/level
`Knowledge.txt`	Knowledge areas required
`Work Context.txt`	Environmental and situational context
`Tools and Technology.txt`	Software, equipment, and tools used

Table 1: Essential O*NET data files

4 Project Components

4.1 Part 1: Framework Development

Deliverable 1: Automation Assessment Framework

Design a structured model for evaluating which jobs/tasks are more likely to be replaced by AI.

4.1.1 Your Task

Develop a framework with 5-8 dimensions that characterize whether a job or task is automatable. For each dimension:

- Define what it measures
- Explain why it matters for automation potential
- Specify how it will be scored (e.g., 1-5 scale, Low/Medium/High)
- Provide examples of low vs. high values

4.1.2 Suggested Dimensions to Consider

Note: You don't have to use all of these. Choose and justify the most important ones.

1. **Routine vs. Non-Routine:**
   - Do tasks follow predictable patterns?
   - Can they be codified into rules?
   - Are they repetitive or variable?
2. **Cognitive Complexity:**
   - Does the task require complex problem-solving?
   - Is deep expertise needed?
   - How much contextual understanding is required?
3. **Physical vs. Digital:**
   - Does the job require physical presence/manipulation?
   - Is it primarily information processing?
   - Can it be done remotely?
4. **Human Interaction Requirements:**
   - Does it require empathy, emotional intelligence?
   - Is building trust/relationships essential?
   - Does it involve negotiation, persuasion, or counseling?
5. **Creativity & Innovation:**
   - Does it require original thinking?
   - Are novel solutions needed?
   - Is aesthetic judgment involved?
6. **Decision-Making Authority:**
   - Are high-stakes decisions made?
   - Is accountability required?
   - How much autonomy is involved?
7. **Data Availability:**
   - Is there structured data to learn from?
   - Can task success be measured objectively?
   - Is the task well-documented?
8. **Current AI Capabilities:**
   - Are AI tools already used for this task?
   - What's the current state of automation?
   - Are there technical barriers?

4.1.3 Framework Example

Here's a partial example to illustrate:

Dimension	Scale	Weight	Rationale
Routine Level	1-5	25%	Routine tasks easier to automate
Physical Req.	1-5	15%	Physical tasks harder for current AI
Social Skills	1-5	20%	Human interaction harder to automate
Creativity	1-5	20%	Creative tasks less automatable
Data Avail.	1-5	20%	Data needed to train AI systems

Table 2: Example framework (you should develop your own)

Automation Score Formula: You'll need to define how to combine these dimensions into an overall automation likelihood score (0-100% or similar).

4.1.4 Key Questions to Answer in Your Documentation

- Why did you choose these specific dimensions?
- How did you determine the weights/importance of each?
- What research or reasoning supports your framework?
- What are the limitations of your framework?
- How did you validate that your dimensions are independent (not measuring the same thing)?

4.2 Part 2: Data Collection and Processing

Deliverable 2: O*NET Data Pipeline

Build a system to extract and structure relevant O*NET data for your analysis.

4.2.1 Your Task

1. **Download and explore O*NET data**
   - Understand the data structure
   - Identify which files contain information relevant to your framework
   - Document any data quality issues
2. **Select occupations to analyze**
   - Choose 20-50 diverse occupations spanning different industries
   - Include variety: high-skill/low-skill, physical/cognitive, creative/routine
   - Document why you selected these occupations
3. **Extract relevant data**
   - Pull task descriptions for each occupation

- Extract skills, abilities, work activities as needed for your framework
- Create a structured dataset (CSV, JSON, or database)

4. **Data preprocessing**
   - Clean and standardize task descriptions
   - Merge data from multiple files
   - Create a unified dataset ready for LLM analysis

4.2.2 Example Code Structure

```python
import pandas as pd

# Load O*NET data files
occupations = pd.read_csv('Occupation Data.txt', sep='\t')
tasks = pd.read_csv('Task Statements.txt', sep='\t')
skills = pd.read_csv('Skills.txt', sep='\t')
abilities = pd.read_csv('Abilities.txt', sep='\t')

# Select diverse occupations
selected_occupations = [
    '15-1252.00',  # Software Developers
    '29-1141.00',  # Registered Nurses
    '25-2021.00',  # Elementary School Teachers
    '53-3032.00',  # Heavy Truck Drivers
    # ... add more
]

# Extract data for selected occupations
occupation_data = {}
for occ_code in selected_occupations:
    occupation_data[occ_code] = {
        'title': occupations[occupations['O*NET-SOC Code'] == occ_code]['Title'].values[0],
        'tasks': tasks[tasks['O*NET-SOC Code'] == occ_code]['Task'].tolist(),
        'skills': skills[skills['O*NET-SOC Code'] == occ_code],
        # ... add more relevant data
    }
```

4.3 Part 3: LLM-Powered Analysis

Deliverable 3: LLM-Based Assessment System

Use LLM APIs to systematically evaluate jobs/tasks according to your framework.

4.3.1 Your Task

Design and implement a prompt engineering strategy that uses LLMs to score jobs and tasks based on your framework dimensions.

Key Requirements:

1. **Structured Prompts:**
   - Create prompts that ask the LLM to evaluate specific dimensions
   - Include clear scoring criteria in the prompt
   - Request structured output (JSON preferred)
   - Include examples to improve consistency
2. **Consistency Testing:**
   - Test the same job multiple times
   - Measure response consistency
   - Adjust prompts to reduce variance
3. **Validation:**
   - Compare LLM assessments to human intuition
   - Check for face validity (do scores make sense?)
   - Identify and document any systematic biases
4. **API Choice:**
   - OpenAI GPT-4 (recommended)
   - Anthropic Claude
   - Google Gemini
   - Or any other LLM API

4.3.2 Prompt Engineering Guidelines

Example Prompt Structure:

1. Routine Level (1-5): How predictable and rule-based is this task? 2. Physical Requirements (1-5): Does it require physical manipulation? 3. Social Skills (1-5): Does it require human empathy/interaction? 4. Creativity (1-5): Does it require novel, creative solutions? 5. Data Availability (1-5): Is there data to train AI on this?
Task: "[INSERT TASK DESCRIPTION]"
Job Context: "[INSERT JOB TITLE AND BRIEF DESCRIPTION]"
Provide your assessment in JSON format: { "routine_level": {score: X, reasoning: "..."}, "physical_req": {score: X, reasoning: "..."}, ... }
Be specific and justify each score.

```
You are an expert in labor economics and AI automation. Analyze the following job task and evaluate its automation potential across these dimensions:
1. Routine Level (1-5): How predictable and rule-based is this task? 2. Physical Requirements (1-5): Does it require physical manipulation? 3. Social Skills (1-5): Does it require human empathy/interaction? 4. Creativity (1-5): Does it require novel, creative solutions? 5. Data Availability (1-5): Is there data to train AI on this?
Task: "[INSERT TASK DESCRIPTION]"
Job Context: "[INSERT JOB TITLE AND BRIEF DESCRIPTION]"
Provide your assessment in JSON format: { "routine_level": {score: X, reasoning: "..."}, "physical_req": {score: X, reasoning: "..."}, ... }
Be specific and justify each score.
```

4.3.3 Implementation Approach

```python
import openai  # or anthropic, google.generativeai, etc.

def assess_task_automation(task_description, job_context,
    framework_dimensions):
    """
    Use LLM to assess automation potential of a task.

    Parameters:
    - task_description: The specific task to analyze
    - job_context: Job title and context
    - framework_dimensions: Your framework dimensions with descriptions

    Returns:
    - Dictionary with scores for each dimension
    """

    # Build prompt based on your framework
    prompt = build_assessment_prompt(task_description, job_context,
        framework_dimensions)

    # Call LLM API
    response = openai.ChatCompletion.create(
        model="gpt-4",
        messages=[
            {"role": "system", "content": "You are an expert in labor
                economics and AI automation."},
            {"role": "user", "content": prompt}
        ],
        temperature=0.3  # Lower temperature for more consistent results
    )

    # Parse response (expecting JSON)
    result = parse_llm_response(response.choices[0].message.content)

    return result

def calculate_automation_score(dimension_scores, weights):
    """
    Calculate overall automation likelihood based on dimension scores.
    """
    # Your formula here
    pass
```

4.3.4 Critical Considerations

Important: LLM Limitations

- LLMs may have biases about certain professions
- They may overestimate or underestimate AI capabilities
- Responses may vary even with same input
- They reflect training data which may be outdated

Your job: Design your framework and prompts to mitigate these issues. Document any biases you observe and how you handled them.

4.4 Part 4: Interactive Application

Deliverable 4: Web Application

Build a user-friendly application where people can explore AI automation risk for different jobs.

4.4.1 Application Requirements

Core Features:

1. **Job Selection Interface:**
   - Search/dropdown to select an occupation
   - Display job title and description
   - Show number of tasks analyzed
2. **Overall Automation Assessment:**
   - Overall automation likelihood score (0-100%)
   - Risk category (Low/Medium/High)
   - Clear explanation of what the score means
3. **Dimension Breakdown:**
   - Show scores for each framework dimension
   - Visualize with bar charts, radar charts, or similar
   - Explain what each dimension means
4. **Task-Level Analysis:**
   - List individual tasks for the job
   - Show automation likelihood for each task
   - Identify which tasks are most/least automatable

5. **Insights and Recommendations:**
   - Which skills to develop to remain relevant
   - Similar jobs with different automation profiles
   - Trends in the occupation's industry

Technical Stack Options:

- **Streamlit (Recommended for beginners):**
  - Python-based, very quick to build
  - Built-in components for charts, selectors
  - Easy deployment
- **Flask/FastAPI + React:**
  - More control over design
  - Better for complex applications
  - Steeper learning curve
- **Gradio:**
  - Very simple for demos
  - Good for quick prototypes

4.4.2 Example Streamlit App Structure

```
import streamlit as st
import pandas as pd
import plotly.graph_objects as go

# Load pre-computed automation scores
@st.cache_data
def load_data():
    return pd.read_csv('occupation_automation_scores.csv')

# Main app
st.title("AI Job Displacement Analysis Tool")

# Job selection
data = load_data()
selected_job = st.selectbox(
    "Select an occupation:",
    data['job_title'].unique()
)

# Display results
job_data = data[data['job_title'] == selected_job].iloc[0]

# Overall score
st.header(f"Automation Risk: {job_data['automation_score']:.0f}%")
```

```
st.subheader(f"Risk Level: {job_data['risk_category']}")

# Dimension breakdown
st.header("Dimension Analysis")
dimensions = ['routine_level', 'physical_req', 'social_skills',
              'creativity', 'data_availability']
scores = [job_data[dim] for dim in dimensions]

# Radar chart
fig = go.Figure(data=go.Scatterpolar(
    r=scores,
    theta=['Routine', 'Physical', 'Social', 'Creativity', 'Data'],
    fill='toself'
))
st.plotly_chart(fig)

# Task-level analysis
st.header("Task Analysis")
tasks_df = load_task_data(selected_job)
st.dataframe(tasks_df[['task_description', 'automation_likelihood']])

# Recommendations
st.header("Career Resilience Recommendations")
st.write(generate_recommendations(job_data))
```

4.5 Part 5: Visualization and Insights

Deliverable 5: Data Visualizations and Analysis

Create compelling visualizations that communicate your findings.

4.5.1 Required Visualizations

1. **Occupation Comparison:**
   - Bar chart comparing automation scores across occupations
   - Group by industry or skill level
   - Identify highest/lowest risk jobs
2. **Dimension Analysis:**
   - Radar/spider chart showing dimension scores for a job
   - Heatmap showing all dimensions across multiple jobs
3. **Task Distribution:**
   - Distribution of task automation scores within a job
   - Show which percentage of tasks are high/medium/low risk
4. **Insights Dashboard:**
   - Summary statistics across all analyzed jobs

- Trends and patterns discovered
- Surprising findings

4.5.2 Visualization Libraries

- **Plotly:** Interactive charts, works well with Streamlit
- **Matplotlib/Seaborn:** Static charts, publication quality
- **Altair:** Declarative visualization
- **D3.js:** For advanced web visualizations

5 Decision Documentation Requirements

Design Decision Log

Just like the recommendation system project, you must document ALL major decisions throughout this project.

Key decisions to document:

1. **Framework Design:**
   - Which dimensions did you choose and why?
   - How did you determine weights?
   - What alternative frameworks did you consider?
   - How did you validate your framework?
2. **Occupation Selection:**
   - Which occupations did you analyze?
   - Why these specific ones?
   - How did you ensure diversity?
3. **Prompt Engineering:**
   - How did you design your prompts?
   - What iterations did you go through?
   - How did you handle inconsistency?
   - Which LLM did you choose and why?
4. **Scoring Methodology:**
   - How do you combine dimension scores?
   - What thresholds define Low/Medium/High risk?
   - How did you validate your scoring?
5. **Application Design:**

- Which tech stack did you choose?
- How did you structure the user interface?
- What features did you prioritize?

6 Deliverables

6.1 Complete Project Package

Your final submission should include:

1. **Documentation (PDF or Markdown):**
   - Project overview and objectives
   - Framework description and justification (2-3 pages)
   - Data collection methodology
   - Prompt engineering strategy and examples
   - Results and findings (3-5 pages)
   - Design decision log (8-10 major decisions)
   - Limitations and future work
   - References and data sources
2. **Code Repository:**
   - Data collection scripts
   - LLM analysis code
   - Application code
   - README with setup instructions
   - Requirements.txt or environment.yml
3. **Data Files:**
   - Cleaned O*NET data subset
   - Automation scores for all analyzed occupations
   - Task-level assessment results
4. **Working Application:**
   - Deployed web app (URL) OR
   - Clear instructions to run locally
   - Demo video (3-5 minutes) showing key features
5. **Presentation (Optional but Recommended):**
   - 10-15 slides
   - Framework overview
   - Key findings
   - Demo of application
   - Insights and recommendations

7 Evaluation Criteria

7.1 Grading Rubric

Component	Weight
Framework Design & Justification	25%
LLM Integration & Prompt Engineering	20%
Application Quality & Usability	20%
Data Analysis & Visualizations	15%
Design Decision Documentation	15%
Code Quality & Documentation	5%
Total	**100%**

7.2 What Makes a Strong Project

Framework Design (25%):

- Well-researched, theoretically grounded dimensions
- Clear justification for each dimension and weight
- Evidence of critical thinking about what makes jobs automatable
- Validation through testing or literature review
- Acknowledgment of limitations

LLM Integration (20%):

- Thoughtful prompt design with clear instructions
- Structured output format (JSON preferred)
- Consistency testing and iteration
- Awareness of LLM biases and limitations
- Appropriate use of AI as a tool, not a black box

Application Quality (20%):

- Intuitive, user-friendly interface
- All required features implemented
- Runs without errors
- Responsive and performant
- Good visual design and layout

Analysis & Visualizations (15%):

- Clear, informative visualizations
- Appropriate chart types for the data
- Insights drawn from analysis
- Comparison across occupations
- Identification of patterns and trends

Decision Documentation (15%):

- 8-10 major decisions documented
- Multiple alternatives considered
- Evidence-based justification
- Clear understanding of trade-offs

Code Quality (5%):

- Clean, readable code
- Proper documentation
- Runs without errors
- Good project organization

8 Getting Started: Suggested Timeline

While you should work at your own pace, here's a suggested breakdown:

Phase 1: Research & Framework Design (3-4 days)

- Research AI automation and job displacement literature
- Explore O*NET database structure
- Design your assessment framework
- Document framework decisions

Phase 2: Data Collection (2-3 days)

- Download O*NET data
- Select occupations to analyze
- Build data extraction pipeline
- Create structured dataset

Phase 3: LLM Analysis (3-4 days)

- Design initial prompts

- Test with sample jobs/tasks
- Iterate on prompt design
- Run analysis on all occupations
- Validate results

Phase 4: Application Development (4-5 days)

- Set up development environment
- Build core features
- Create visualizations
- Test with users
- Polish and deploy

Phase 5: Documentation & Refinement (2-3 days)

- Write comprehensive documentation
- Complete decision log
- Create presentation
- Final testing and bug fixes

Total: 14-19 days

9 Technical Resources

9.1 O*NET Resources

- **Main Database:** https://www.onetcenter.org/database.html
- **Data Dictionary:** https://www.onetcenter.org/dictionary/
- **API Documentation:** https://services.onetcenter.org/reference/
- **Occupation Finder:** https://www.onetonline.org/

9.2 LLM API Resources

- **OpenAI:** https://platform.openai.com/docs/
- **Anthropic Claude:** https://docs.anthropic.com/
- **Google Gemini:** https://ai.google.dev/

9.3 Application Development

- **Streamlit:** https://docs.streamlit.io/
- **Plotly:** https://plotly.com/python/
- **Pandas:** https://pandas.pydata.org/docs/

9.4 Background Reading

- Frey & Osborne (2017): "The Future of Employment"
- Autor, Levy, Murnane (2003): "The Skill Content of Recent Technological Change"
- Brynjolfsson & Mitchell (2017): "What Can Machine Learning Do?"
- Webb (2020): "The Impact of Artificial Intelligence on the Labor Market"

10 Important Considerations

10.1 Ethical Considerations

Think Critically About Impact

Your tool will help people understand AI's impact on their careers. Consider:

- How might your assessment affect someone's career decisions?
- Are there biases in how you're measuring automation potential?
- What's your responsibility in presenting these findings?
- How do you balance honesty with not creating undue panic?

Include an "About" section in your app explaining:

- What your scores mean and don't mean
- Limitations of the analysis
- That this is one perspective, not absolute truth
- Encouragement to focus on developing adaptable skills

10.2 AI as Tool vs. Crutch

Remember:

- LLMs help you analyze systematically, but YOUR framework drives the analysis
- The quality of your results depends on the quality of your framework and prompts
- AI doesn't replace your critical thinking—it amplifies it
- Document where AI helps and where human judgment is essential

10.3 Dealing with Uncertainty

- Your assessments are informed estimates, not predictions
- Technology evolves rapidly—what's true today may not be tomorrow
- Different experts have different views on automation potential
- Be honest about uncertainty in your documentation and app

11 Example Projects for Inspiration

11.1 Possible Extensions (Optional)

If you finish early or want to go beyond requirements:

1. **Skill Gap Analysis:**
   - Identify skills workers should develop
   - Recommend training resources
2. **Career Transition Recommendations:**
   - Suggest similar jobs with lower automation risk
   - Show skill transferability between occupations
3. **Temporal Analysis:**
   - Estimate timeline for automation (near-term vs. long-term)
   - Track how assessments change with new AI capabilities
4. **Industry-Level Insights:**
   - Analyze entire industries
   - Identify sectors most/least affected
5. **Comparative Analysis:**
   - Compare your LLM-based assessments to existing research
   - Validate against expert opinions

12 Common Pitfalls to Avoid

1. **Treating LLM output as ground truth**
   - LLMs can be wrong or biased
   - Always validate and cross-check
2. **Oversimplifying complex jobs**
   - Jobs aren't monolithic—task-level analysis is important
   - Automation job elimination (augmentation vs. replacement)

3. **Ignoring current AI limitations**
    - Physical manipulation is still hard
    - Common sense reasoning is challenging
    - Human relationships require empathy
4. **Poor prompt design**
    - Vague prompts lead to inconsistent results
    - Always include concrete examples and criteria
5. **Confusing correlation with causation**
    - Just because tasks are routine doesn't mean they'll be automated
    - Economic and organizational factors matter too
6. **Building an app without user testing**
    - Test with real users
    - Iterate based on feedback

13 Questions & Support

When asking for help:

- Share what you've tried
- Show specific code or examples
- Explain your thinking and where you're stuck

Good questions:

- "I'm trying to decide between dimension X and Y. I tested both and here are the results. How should I think about this trade-off?"
- "My LLM is giving inconsistent scores. I tried lowering temperature and adding examples. What else should I try?"
- "I want to weight creativity higher than routine. Is there research to support this?"

This project combines AI, data science, and real-world impact. Take it seriously, think critically, and build something meaningful. Good luck!